

Sprache: Python 3 (pandas, NumPy, SciPy, SQLAlchemy, FastAPI, yfinance) · Repository:
github.com/phile2006/Portfolio-Optimizer

1 – Ziel des Projekts

Aufbauend auf der [Stock-Correlation-Heatmap](#) (Projekt 2) soll ein **Mean-Variance-Optimizer nach Markowitz** entstehen: Für eine Liste von Aktien/ETFs werden historische Kurse geladen, daraus erwartete Renditen und die Kovarianzmatrix geschätzt, und anschließend die **optimalen Portfoliogewichte** berechnet — das Minimum-Varianz-Portfolio, das Maximum-Sharpe-Portfolio (Tangency) und die Efficient Frontier. Die Korrelations-/Kovarianzmatrix, die Projekt 2 nur *visualisiert*, wird hier zum zentralen *Rechen-Input*.

Anders als die beiden Vorgängerprojekte ist dies bewusst **kein reines CLI-Tool**: Das Projekt führt zwei neue Bausteine ein — eine **SQLite-Datenbank** (Persistenz von Kursen und Optimierungsläufen) und eine **REST-API mit FastAPI**. Damit deckt es genau die Inhalte der Module „Datenmodellierung und Datenbanksysteme“ und „Angewandte Programmierung“ praktisch ab.

2 – Fachlicher Hintergrund

Mean-Variance-Optimierung (Markowitz)

Aus täglichen Renditen werden der annualisierte Erwartungswert-Vektor μ ($\times 252$ Handelstage) und die annualisierte Kovarianzmatrix Σ geschätzt. Ein Portfolio ist ein Gewichtsvektor w ; seine Kennzahlen:

$$\text{Rendite: } w^T \mu \quad \cdot \quad \text{Volatilität: } \sqrt{(w^T \Sigma w)} \quad \cdot \quad \text{Sharpe: } (w^T \mu - r_f) / \sqrt{(w^T \Sigma w)}$$

Nebenbedingungen in diesem Projekt: **long-only** ($w_i \geq 0$) und **voll investiert** ($\sum w_i = 1$). Gelöst wird numerisch mit SLSQP aus `scipy.optimize`. Drei Zielfunktionen:

- **Minimum Variance**: minimiere $w^T \Sigma w$ — das defensivste Portfolio.
- **Maximum Sharpe**: maximiere die Sharpe Ratio — bestes Rendite-Risiko-Verhältnis.
- **Efficient Frontier**: minimiere die Varianz für jede Zielrendite zwischen dem rendite-schwächsten und -stärksten Einzelwert; die Punkte ergeben die Frontier-Kurve.

Bekannte Schwäche des Ansatzes: Die Optimierung reagiert sehr sensibel auf die geschätzten μ — historische Mittelwerte sind schlechte Prognosen. Das gehört ins README, nicht in den Code.

Warum Datenbank und API?

Datenbank: Kurse werden einmal geladen und wiederverwendet statt bei jedem Lauf neu; zusätzlich wird jeder Optimierungslauf mit Ergebnis gespeichert — es entsteht eine abfragbare Historie („was habe ich wann mit welchem Ergebnis optimiert?“). **API**: Die Rechenlogik wird über HTTP nutzbar — die Vorstufe zu jedem echten Frontend und der Standard-Integrationsweg in Unternehmen.

3 – Architektur-Vorgabe

Drei Schichten, jede mit genau einer Verantwortung; der Kern kennt weder Datenbank noch HTTP:

Schicht	Datei	Verantwortung
Daten	portfolio_optimizer/data.py	Adjusted-Close-Kurse von Yahoo Finance laden
Persistenz	portfolio_optimizer/database.py	Kurse + Läufe speichern/lesen (SQLite, SQLAlchemy)
Kern	portfolio_optimizer/optimizer.py	numerische Markowitz-Optimierung (pur)
API	portfolio_optimizer/api.py	Kern + DB über HTTP verfügbar machen (FastAPI)
CLI	portfolio_optimizer/cli.py	schlanker Kommandozeilen-Einstieg (ohne DB)

4 – Aufgabenstellung (in Stufen — jede Stufe läuft für sich)

Stufe 1 — CLI + Rechenkern

- Implementiere den Kern: Renditen $\rightarrow$ annualisierte $\mu/\Sigma \rightarrow$ `min_variance`, `max_sharpe`, `efficient_frontier(points)`; Rückgabe als Datenobjekt (Gewichte, erwartete Rendite, Volatilität, Sharpe).
- Validierung: mindestens 2 Ticker, keine Lücken in der Kursmatrix, unbekannte Zielfunktion $\Rightarrow$ Fehler.
- CLI: `python -m portfolio_optimizer.cli AAPL MSFT GLD TLT --objective max_sharpe --period 2y` druckt Gewichte (sortiert) und Kennzahlen.

Stufe 2 — Datenbank

SQLite (`portfolio.db`) über SQLAlchemy mit diesem Schema:

Tabelle	Spalten
securities	ticker (PK), name
prices	ticker (FK $\rightarrow$ securities), date, close; PK (ticker, date)
optimization_runs	id (PK), created_at, objective, tickers (JSON), weights (JSON), expected_return, volatility, sharpe

- Funktionen: `save_prices` (Upsert), `load_prices` (zurück als breite Matrix), `save_run`, `get_run`, `list_securities`.

Stufe 3 — Web-API

Methode	Pfad	Zweck
POST	/optimize	Kurse laden, persistieren, optimieren, Lauf speichern und zurückgeben
GET	/securities	gespeicherte Wertpapiere auflisten
GET	/runs/{id}	gespeicherten Optimierungslauf abrufen
GET	/frontier	Frontier-Punkte für eine Tickerliste

Beispiel — Request und Response für POST /optimize:

```
{ "tickers": ["AAPL", "MSFT", "GLD", "TLT"], "period": "2y", "objective":  
  "max_sharpe" }
```

```
{ "run_id": 1, "objective": "max_sharpe",  
  "weights": { "AAPL": 0.41, "MSFT": 0.22, "GLD": 0.37, "TLT": 0.0 },  
  "expected_return": 0.18, "volatility": 0.14, "sharpe": 1.29 }
```

- Eingaben mit Pydantic validieren (≥ 2 Ticker); fachliche Fehler als HTTP 400, unbekannte Run-ID als 404.
- Interaktive Doku unter /docs (bekommt FastAPI geschenkt — im Vorstellungsgespräch zeigen!).

Stufe 4 — Frontend (geplant)

- Kleine HTML/JS-Seite, die /frontier aufruft und die Kurve zeichnet (Volatilität x, Rendite y); Min-Var- und Max-Sharpe-Portfolio als Punkte markieren.

5 – Technische Anforderungen

- Tests mit `pytest` auf **synthetischen Kursen** (kein Netzwerk nötig): Der Rechenkern muss ohne Internet testbar sein.
- Fail-fast-Fehlerbehandlung: ungültige Eingaben führen zu klaren Exceptions, keine stillen Defaults.
- `requirements.txt`, `venv`-Setup, `.gitignore` (u.a. `*.db`).

6 – Abnahmekriterien & Verifikation

Das Projekt gilt als fertig, wenn:

- **Gewichte sind gültig:** Summe = 1, alle ≥ 0 (long-only) — für jede Zielfunktion.
- **Min-Varianz schlägt Gleichgewichtung:** Die Volatilität des Minimum-Varianz-Portfolios ist nie höher als die des $1/n$ -Portfolios.
- **Frontier ist konsistent:** Renditen entlang der Frontier steigen monoton; kein Frontier-Punkt hat eine höhere Sharpe Ratio als das Tangency-Portfolio.
- **Persistenz funktioniert:** Nach POST /optimize liefert GET /runs/{id} denselben Lauf zurück; `load_prices` reproduziert die gespeicherte Kursmatrix.
- Alle `pytest`-Tests laufen grün, ohne Netzwerkzugriff.

7 – Mögliche Erweiterungen

- Frontend (Stufe 4) mit Plotly/Chart.js; Klick auf einen Frontier-Punkt zeigt dessen Gewichte.
- Shrinkage-Schätzer für Σ (Ledoit-Wolf) — stabilere Gewichte bei vielen Assets.
- Constraints: maximales Einzelgewicht (z. B. $\leq 30\%$) oder Sektor-Limits.
- Backtest: optimierte Gewichte auf einem späteren Zeitraum evaluieren (out-of-sample).